# ISLS Extension Phase 6 v1.0.0

## C25: Hybrid Synthesis Oracle

LLM-Bridge with Progressive Pattern Autonomy

The System That Learns to Stop Asking

Sebastian Klemm

17 March 2026

**Abstract**

This document specifies `isls-oracle` (C25), a hybrid synthesis layer that bridges the ISLS Forge (C21–C24) to external Large Language Models for artifact generation, while progressively building internal pattern autonomy.

The mechanism: when the Forge needs to synthesize concrete content (code, specs, schemas), it first checks the Pattern Memory for a sufficiently similar crystallized pattern. If a match is found with quality above threshold, the pattern is adapted and used directly — no LLM call. If no match exists, the system constructs a maximally-constrained prompt from the ArtifactIR (components, interfaces, constraints, quality goals), sends it to the LLM, receives the output, validates it through the full 8-gate cascade, and — if it passes — crystallizes it into the Pattern Memory for future reuse.

Over time, as the Pattern Memory grows with validated patterns, the LLM is called less and less. The system measures its own *autonomy ratio*: the fraction of synthesis requests served from memory without LLM involvement. The long-term trajectory is asymptotic independence.

The LLM is never trusted. It is an oracle whose output is treated as an unverified hypothesis that must survive adversarial quality checking (PMHD) and formal validation (8 gates) before becoming a crystal. The system's guarantees derive from ISLS, not from the LLM.

**Status:** Implementation Specification.

**Parent:** ISLS v1.0.0 + Extensions Phase 1–5.1 + Genesis.

**Depends on:** C14 (Capsule, for API key protection), C19 (Gateway), C21 (PMHD), C22 (ArtifactIR), C23 (Forge), C24 (Compose).

# Contents

I Foundations 5

1 The Problem 5

2 Axioms 5

II Architecture 5

3 The Synthesis Decision Tree 5

4 The Oracle Trait 6

5 Built-In Oracle: Claude 8

6 Prompt Engineering 8

III Pattern Memory Integration 9

7 Memory-First Lookup 10

8 Autonomy Tracking 10

IV Validation Pipeline for Oracle Output 11

9 Multi-Stage Validation 11

V Security and Governance 12

10 Capsule-Protected API Key 12

11 Budget Controls 12

12 Prompt Sanitization 12

VI Configuration and API 13

13 Configuration 13

14 Rust API 13

15 Integration with Forge 14

16 CLI Integration 15

17 Gateway Integration 15

**VII The Autonomy Curve** **16**

**18 How the System Learns Independence** **16**

**VIII Acceptance Tests** **16**

**IX Handoff** **17**

**19 Dependencies** **17**

**20 Test Count** **18**

**21 Completion Criteria** **18**

# Part I
# Foundations

## 1 The Problem

The Forge (C21–C24) has the complete pipeline: DecisionSpec → PMHD Drill → ArtifactIR → Matrix → Synth → Eval → Crystal. But the Synth step — where abstract component descriptions become concrete code, specifications, or schemas — produces structural skeletons, not functional artifacts. The system knows *what* needs to exist (components, interfaces, constraints) but cannot generate *how* it works (implementation logic, algorithm choice, API design).

An LLM can generate the "how." But an LLM cannot prove its output is correct, consistent, or reproducible. ISLS can prove those things but cannot generate creative content.

The hybrid approach: let the LLM generate, let ISLS validate, and let every validated result become a pattern that reduces future LLM dependency.

## 2 Axioms

**Axiom 2.1** (Oracle Distrust)**.** The LLM is an untrusted external oracle. Its output is treated as a raw hypothesis, never as a validated artifact. Every LLM output **MUST** pass PMHD adversarial checking and the 8-gate cascade before crystallization. No LLM output enters the archive unchecked.

**Axiom 2.2** (Memory First)**.** The Pattern Memory is always consulted before the LLM. If a crystallized pattern with sufficient similarity and quality exists, it is used. The LLM is a fallback, not a default.

**Axiom 2.3** (Progressive Autonomy)**.** The system **MUST** track and report its autonomy ratio: the fraction of synthesis requests served from Pattern Memory without LLM involvement. The design goal is monotonic growth of this ratio over time.

**Axiom 2.4** (Deterministic Envelope)**.** The LLM call itself is non-deterministic (different runs may produce different outputs). But the ISLS envelope is deterministic: the prompt construction is deterministic (same ArtifactIR → same prompt), the validation is deterministic, and the crystallization is deterministic. The non-determinism is confined to the oracle boundary and is absorbed by the validation gate.

**Axiom 2.5** (Graceful Degradation)**.** If the LLM is unavailable (no API key, network down, rate limited), the system **MUST** fall back to Pattern Memory only. If Pattern Memory has no match, the system emits a structural skeleton (as today) rather than failing. The system never depends on LLM availability for correctness.

# Part II
# Architecture

## 3 The Synthesis Decision Tree

Synthesis Decision Flow

```
ForgeEngine receives ArtifactIR for synthesis
    |
    v
[1] Query Pattern Memory
    |-- Match found (similarity >= theta_match, quality >= theta_q)?
    |       |
    |       YES --> [2] Adapt pattern to current ArtifactIR
    |               |
    |               v
    |               [3] Validate adapted output (8-gate)
    |               |-- PASS --> Emit crystal (source: "memory")
    |               |-- FAIL --> Fall through to LLM path
    |
    NO (or memory adaptation failed)
    |
    v
[4] Check LLM availability
    |-- LLM unavailable?
    |       |
    |       YES --> [5] Emit structural skeleton (source: "skeleton")
    |
    NO
    |
    v
[6] Construct prompt from ArtifactIR
    |
    v
[7] Call LLM oracle
    |
    v
[8] Parse and validate LLM response
    |-- Parse failure? --> Retry (up to max_retries)
    |-- Parse success:
    |
    v
[9] PMHD adversarial check on LLM output
    |
    v
[10] 8-gate validation
    |-- FAIL --> Retry with refined prompt (up to max_retries)
    |-- PASS:
    |
    v
[11] Crystallize and store in Pattern Memory
    |   (source: "oracle", oracle_model: "claude-sonnet-4-20250514")
    |
    v
[12] Emit crystal
```

# 4 The Oracle Trait

**Definition 4.1** (SynthesisOracle Trait)**.** Every external synthesis provider implements:

```
pub trait SynthesisOracle: Send + Sync {
    /// Human-readable name
    fn name(&self) -> &str;

    /// Model identifier
    fn model(&self) -> &str;

    /// Check availability (API key present, endpoint reachable)
    fn available(&self) -> bool;

    /// Generate synthesis output from a structured prompt
    fn synthesize(
        &self,
        prompt: &SynthesisPrompt,
    ) -> Result<OracleResponse, OracleError>;

    /// Estimated cost per call (for budget tracking)
    fn cost_estimate(&self) -> OracleCost;
}
```

**Definition 4.2** (SynthesisPrompt)**.** The prompt sent to the oracle is fully determined by the ArtifactIR:

```
pub struct SynthesisPrompt {
    /// System instruction (role, constraints, output format)
    pub system: String,
    /// The actual request (derived from ArtifactIR)
    pub user: String,
    /// Expected output format
    pub output_format: OutputFormat,
    /// Maximum tokens
    pub max_tokens: usize,
    /// Temperature (0.0 for maximum determinism)
    pub temperature: f64,
}

pub enum OutputFormat {
    Rust,
    Json,
    Yaml,
    OpenApi,
    PlainText,
}
```

**Definition 4.3** (OracleResponse)**.**

```
pub struct OracleResponse {
    pub content: String,          // raw LLM output
    pub model: String,            // model that produced it
    pub tokens_used: usize,       // for budget tracking
    pub finish_reason: String,    // "stop", "length", etc.
    pub latency_ms: u64,
}
```

**Invariant 4.1** (Prompt Determinism)**.** The prompt construction from ArtifactIR **MUST** be deterministic: identical ArtifactIR produces identical SynthesisPrompt. The LLM's response is non-deterministic, but the question is always the same.

# 5 Built-In Oracle: Claude

**Definition 5.1** (ClaudeOracle)**.** The default built-in oracle implementation uses the Anthropic API:

```
pub struct ClaudeOracle {
    api_key: String,          // from Capsule or env var
    model: String,            // default: "claude-sonnet-4-20250514"
    endpoint: String,         // https://api.anthropic.com/v1/messages
    max_retries: usize,       // default: 3
    timeout_ms: u64,          // default: 60000
    temperature: f64,         // default: 0.0
}
```

The API key **MAY** be stored in an ISLS Capsule (C14), released only when the system is in a valid constitutional state (Genesis Crystal valid). This means: if the system's constitution is violated, it loses access to the LLM. Self-enforcing governance.

**Requirement 5.1** (API Key Security)**.** The API key **MUST NOT** appear in any log, trace, crystal, manifest, or report. It **MUST NOT** be part of the SynthesisPrompt serialization. It is used only in the HTTP request header and is never persisted in ISLS storage.

# 6 Prompt Engineering

**Definition 6.1** (Prompt Construction)**.** The prompt is constructed from the ArtifactIR by a deterministic `PromptBuilder`:

```
pub struct PromptBuilder;

impl PromptBuilder {
    pub fn build(
        ir: &ArtifactIR,
        matrix: &dyn Matrix,
        config: &OracleConfig,
    ) -> SynthesisPrompt {
        let system = format!(
            "You are a code synthesis engine. You produce ONLY \
             valid {} code. No explanations, no markdown fences, \
             no commentary. Output ONLY the requested artifact.\n\n\
             CONSTRAINTS:\n{}\n\n\
             QUALITY REQUIREMENTS:\n\
             - Coherence >= {}\n\
             - Robustness >= {}\n\
             - All interfaces must be satisfied\n\
             - Code must compile (if applicable)\n\n\
             OUTPUT FORMAT: {}",
            matrix.domain(),
            ir.constraints.join("\n"),
            ir.metrics.coherence,
            ir.metrics.robustness,
            config.output_format.as_str(),
        );

        let user = Self::build_user_prompt(ir, matrix);
```

```
        SynthesisPrompt {
            system,
            user,
            output_format: config.output_format.clone(),
            max_tokens: config.max_tokens,
            temperature: config.temperature,
        }
    }

    fn build_user_prompt(ir: &ArtifactIR, matrix: &dyn Matrix) -> String
        {
        let mut parts = Vec::new();

        // Component descriptions
        parts.push("COMPONENTS:".to_string());
        for comp in &ir.components {
            parts.push(format!(
                "- {} (kind: {}): {}",
                comp.name, comp.kind, comp.content
            ));
            if !comp.dependencies.is_empty() {
                parts.push(format!(
                    "  depends on: {}", comp.dependencies.join(", ")
                ));
            }
        }

        // Interfaces
        if !ir.interfaces.is_empty() {
            parts.push("\nINTERFACES:".to_string());
            for iface in &ir.interfaces {
                parts.push(format!(
                    "- {} provides to {}: {}",
                    iface.provider, iface.consumer, iface.contract
                ));
            }
        }

        // Provenance (for context)
        parts.push(format!(
            "\nPROVENANCE: monolith {}, drill ticks {}-{}, \
             quality coh={:.2} rob={:.2} cov={:.2}",
            ir.provenance.monolith_id,
            ir.provenance.tick_range[0],
            ir.provenance.tick_range[1],
            ir.metrics.coherence,
            ir.metrics.robustness,
            ir.metrics.coverage,
        ));

        parts.join("\n")
    }
}
```

# Part III
# Pattern Memory Integration

## 7 Memory-First Lookup

**Definition 7.1** (Pattern Match)**.** A pattern $p$ in memory matches an ArtifactIR $a$ if:

$$\text{sim}(p.\text{signature}, a.\text{signature}) \geq \theta_{\text{match}} \ \wedge \ p.\text{quality_score} \geq \theta_q \ \wedge \ p.\text{domain} = a.\text{domain}$$

where sim is cosine similarity in the 5D embedding space, $\theta_{\text{match}}$ is the match threshold (default: 0.85), and $\theta_q$ is the minimum quality (default: 0.6).

**Definition 7.2** (Pattern Adaptation)**.** When a pattern matches but is not identical, it must be adapted to the current ArtifactIR. Adaptation consists of:

(i) **Name substitution**: replace component names from the pattern with names from the current ArtifactIR.
(ii) **Interface alignment**: verify that the pattern's interfaces are compatible with the current interface contracts; adjust signatures if needed.
(iii) **Constraint overlay**: apply any additional constraints from the current ArtifactIR that the pattern doesn't cover.
(iv) **Re-validation**: the adapted output goes through the evaluation stack and 8-gate cascade.

If adaptation produces a valid crystal, it is emitted with `source: "memory_adapted"`.

**Definition 7.3** (Memory Crystallization)**.** When an LLM-generated output passes all validation:

(i) The output is crystallized as a Semantic Crystal with `source: "oracle"` and `oracle_model` in metadata.
(ii) A PatternEntry is created from the crystal: domain, quality metrics, 5D signature, component kinds, the actual content (code/spec/schema).
(iii) The PatternEntry is persisted to the store (C17).
(iv) Future requests with similar signatures will find this pattern in memory-first lookup.

## 8 Autonomy Tracking

**Definition 8.1** (Autonomy Metrics)**.** The oracle crate tracks:

```
pub struct AutonomyMetrics {
    /// Total synthesis requests
    pub total_requests: u64,
    /// Served from pattern memory (no LLM)
    pub memory_hits: u64,
    /// Served from LLM (memory miss)
    pub oracle_calls: u64,
    /// LLM output that failed validation
    pub oracle_rejections: u64,
    /// Fell back to skeleton (LLM unavailable + no memory match)
    pub skeleton_fallbacks: u64,
    /// Autonomy ratio: memory_hits / total_requests
    pub autonomy_ratio: f64,
    /// Total tokens consumed by LLM calls
    pub total_tokens: u64,
    /// Total estimated cost
```

```
    pub total_cost_usd: f64,
}
```

**Definition 8.2** (M33 Autonomy Ratio)**.** A new metric:

$$M_{33} = \frac{\text{memory_hits}}{\text{total_requests}}$$

Range $[0, 1]$. A value of 0 means fully LLM-dependent. A value of 1 means fully autonomous. The metric is reported in the HTML dashboard and tracked as a time series.

**Definition 8.3** (M34 Oracle Rejection Rate)**.**

$$M_{34} = \frac{\text{oracle_rejections}}{\text{oracle_calls}}$$

Measures how often the LLM produces output that fails ISLS validation. High values indicate the LLM is not suited for the domain or the constraints are too strict.

# Part IV
# Validation Pipeline for Oracle Output

## 9 Multi-Stage Validation

LLM output goes through four validation stages before crystallization:

**Definition 9.1** (Stage 1: Parse Validation)**.** The raw LLM output string is parsed according to the expected OutputFormat:

- **Rust**: check that the output is syntactically valid Rust (using `syn::parse_file` or a lightweight syntax check).
- **JSON**: check valid JSON and match against expected schema.
- **YAML**: check valid YAML and match against expected schema.
- **OpenAPI**: check valid OpenAPI 3.0 structure.

If parse fails, retry with a refined prompt (append the parse error to the prompt).

**Definition 9.2** (Stage 2: Constraint Satisfaction)**.** Check that all hard constraints from the DecisionSpec and ArtifactIR are satisfied:

- All required components are present.
- All required interfaces are implemented.
- No forbidden patterns are present.
- Output length is within bounds.

**Definition 9.3** (Stage 3: PMHD Adversarial Check)**.** The LLM output is treated as a hypothesis and run through a mini-PMHD cycle (configurable ticks, default 50):

- Counterexamples are generated against the output.
- Quality metrics (coherence, robustness, etc.) are computed.
- The output must survive the commit gate.

This catches subtle issues: inconsistent naming, missing error handling, interface mismatches that parse validation doesn't catch.

**Definition 9.4** (Stage 4: 8-Gate Crystal Validation)**.** The validated output is packaged as a Semantic Crystal and run through the standard 8-gate cascade. This is the final arbiter.

**Requirement 9.1** (Retry Budget)**.** The system **MUST** have a configurable retry budget (default: 3 attempts per synthesis request). Each retry appends the validation failure reason to the prompt. If the budget is exhausted, the system falls back to skeleton output.

# Part V
# Security and Governance

## 10 Capsule-Protected API Key

**Definition 10.1** (Oracle Key Capsule)**.** The LLM API key **MAY** be stored in an ISLS Capsule (C14) whose policy requires:

- Genesis Crystal is valid (no constitutional drift),
- The requesting run has a valid manifest,
- The operator has not exceeded the budget.

This means: if someone tampers with the system's constitution, the API key becomes inaccessible. Self-enforcing.

## 11 Budget Controls

**Definition 11.1** (Oracle Budget)**.**

```
pub struct OracleBudget {
    /// Maximum LLM calls per run
    pub max_calls_per_run: u64,          // default: 100
    /// Maximum total tokens per run
    pub max_tokens_per_run: u64,         // default: 500_000
    /// Maximum estimated cost per run (USD)
    pub max_cost_per_run: f64,           // default: 10.0
    /// Maximum calls per day (across all runs)
    pub max_calls_per_day: u64,          // default: 1000
    /// Current usage
    pub current: OracleUsage,
}

pub struct OracleUsage {
    pub calls_this_run: u64,
    pub tokens_this_run: u64,
    pub cost_this_run: f64,
    pub calls_today: u64,
}
```

When any budget limit is reached, the system falls back to memory-only or skeleton mode. No silent overspend.

## 12 Prompt Sanitization

**Requirement 12.1** (No Secret Leakage)**.** The PromptBuilder **MUST NOT** include in the prompt:

- API keys, passwords, or cryptographic material,
- Capsule contents or master keys,
- Raw observation data (only aggregate descriptions),

- User-identifiable information.

The prompt contains only: component descriptions, interface contracts, constraints, quality goals, and domain metadata. All derived from the ArtifactIR which itself is content-addressed and auditable.

# Part VI
# Configuration and API

## 13 Configuration

Oracle Configuration

```
oracle:
  enabled: true
  provider: "claude"
  model: "claude-sonnet-4-20250514"
  endpoint: "https://api.anthropic.com/v1/messages"
  api_key_source: "env:ANTHROPIC_API_KEY"  # or "capsule:<id>"
  temperature: 0.0
  max_tokens: 4096
  max_retries: 3
  timeout_ms: 60000
  memory_first: true
  match_threshold: 0.85        # cosine similarity for pattern match
  quality_threshold: 0.6       # minimum pattern quality
  pmhd_validation_ticks: 50    # mini-PMHD on LLM output
  budget:
    max_calls_per_run: 100
    max_tokens_per_run: 500000
    max_cost_per_run: 10.0
    max_calls_per_day: 1000
  fallback: "skeleton"          # "skeleton" or "error"
```

## 14 Rust API

```
// isls-oracle/src/lib.rs

pub struct OracleEngine {
    oracle: Box<dyn SynthesisOracle>,
    memory: PatternMemory,
    prompt_builder: PromptBuilder,
    validator: OracleValidator,
    budget: OracleBudget,
    metrics: AutonomyMetrics,
    config: OracleConfig,
}

impl OracleEngine {
    pub fn new(
        config: OracleConfig,
        memory: PatternMemory,
```

```
    ) -> Self;

    /// The main synthesis method.
    /// Returns the synthesis output and its source.
    pub fn synthesize(
        &mut self,
        ir: &ArtifactIR,
        matrix: &dyn Matrix,
    ) -> Result<OracleSynthesisResult>;

    /// Check if LLM is available
    pub fn oracle_available(&self) -> bool;

    /// Get current autonomy metrics
    pub fn autonomy(&self) -> &AutonomyMetrics;

    /// Get budget status
    pub fn budget_status(&self) -> &OracleBudget;
}

pub struct OracleSynthesisResult {
    pub output: SynthesisOutput,
    pub source: SynthesisSource,
    pub validation: ValidationSummary,
    pub retries_used: usize,
    pub tokens_used: usize,
}

pub enum SynthesisSource {
    /// Served from pattern memory, no LLM call
    Memory { pattern_id: Hash256, similarity: f64 },
    /// Adapted from pattern memory, no LLM call
    MemoryAdapted { pattern_id: Hash256, similarity: f64 },
    /// Generated by LLM oracle
    Oracle { model: String, tokens: usize },
    /// Structural skeleton (LLM unavailable + no memory match)
    Skeleton,
}

pub struct ValidationSummary {
    pub parse_ok: bool,
    pub constraints_ok: bool,
    pub pmhd_ok: bool,
    pub gates_ok: bool,
    pub quality: QualityMetrics,
}
```

## 15 Integration with Forge

Modified ForgeEngine (C23)

```
// In isls-forge, the synthesize step now delegates to OracleEngine:

impl ForgeEngine {
```

```
    pub fn forge(&mut self, spec: DecisionSpec) -> Result<ForgeResult> {
        // ... PMHD drill, ArtifactIR construction ...

        // NEW: use OracleEngine instead of bare Synthesizer
        let synth_result = self.oracle_engine.synthesize(
            &artifact_ir,
            &self.get_matrix(&spec.domain)?,
        );

        match synth_result {
            Ok(result) => {
                // Validate through 8-gate cascade
                // If passed: crystallize, store pattern, emit
                // Source is tracked in crystal metadata
            }
            Err(e) => {
                // Budget exceeded or all retries failed
                // Fall back based on config
            }
        }
    }
}
```

## 16 CLI Integration

```
# Forge with oracle (default if configured)
isls forge --spec intent.json --domain rust

# Forge with memory only (no LLM calls)
isls forge --spec intent.json --domain rust --memory-only

# Forge with skeleton only (no LLM, no memory)
isls forge --spec intent.json --domain rust --skeleton-only

# Show autonomy metrics
isls oracle status
# -> Autonomy: 73.2% (41/56 from memory)
# -> Oracle calls: 15 | Rejections: 2 (13.3%)
# -> Tokens used: 12,450 | Cost: $0.24
# -> Budget remaining: 85/100 calls, 487,550/500,000 tokens

# Show pattern memory stats
isls oracle memory
# -> Patterns: 47 | Domains: rust(23), api(14), schema(10)
# -> Avg quality: 0.74 | Newest: 2h ago

# Seal API key in a capsule (optional security)
isls oracle seal-key --key "sk-ant-..." --lock-genesis
```

## 17 Gateway Integration

New Gateway Endpoints

```
GET  /oracle/status     -> AutonomyMetrics + BudgetStatus
GET  /oracle/memory     -> PatternMemory stats
POST /oracle/synthesize -> Direct synthesis request
                           Body: {ir, domain, memory_only: bool}
```

# Part VII
# The Autonomy Curve

## 18 How the System Learns Independence

1. **Day 1**: Pattern Memory is empty. Every synthesis request goes to the LLM. Autonomy ratio: 0%.
2. **Week 1**: After 50 forge runs across 3 domains (Rust, API, Schema), the memory has ∼40 validated patterns. Simple requests (health checks, CRUD endpoints, config schemas) hit memory. Autonomy: ∼30%.
3. **Month 1**: Memory has ∼200 patterns. Common patterns for auth modules, database layers, validation logic, error handling are crystallized. Most routine synthesis is memory-served. Autonomy: ∼60%.
4. **Month 6**: Memory has ∼1000 patterns. Domain-specific patterns cover most standard architectures. LLM is called only for novel or unusual requests. Autonomy: ∼85%.
5. **Year 1**: Memory is a comprehensive library of validated patterns. LLM calls are rare — only for genuinely new domains or unprecedented combinations. Autonomy: ∼95%.

The curve is asymptotic. The system will likely never reach 100% because novel requests will always exist. But the dependency on external oracles monotonically decreases.

*Remark* 18.1. This is the architectural equivalent of an apprentice who starts by asking the master for every answer, gradually learns the common patterns, and eventually only consults the master for genuinely new problems. The difference: every answer the master gives is formally verified before the apprentice internalizes it.

# Part VIII
# Acceptance Tests

| ID | Name | Procedure |
|---|---|---|
| AT-O1 | Memory-first hit | Seed memory with a pattern; request synthesis with matching signature; verify no LLM call made, source is "memory." |
| AT-O2 | LLM fallback | Empty memory; request synthesis; verify LLM called, output validated, pattern stored. |

| | | |
|---|---|---|
| AT-O3 | Validation rejection | Mock LLM to return invalid code; verify rejection, retry, eventual skeleton fallback. |
| AT-O4 | Prompt determinism | Build prompt from same ArtifactIR twice; verify identical prompt strings. |
| AT-O5 | Budget enforcement | Set max_calls=1; make 2 requests; verify second request falls back to skeleton. |
| AT-O6 | Autonomy tracking | Make 3 memory hits and 2 LLM calls; verify autonomy ratio = 0.6. |
| AT-O7 | Pattern crystallization | LLM produces valid output; verify pattern stored in memory and retrievable by signature. |
| AT-O8 | Graceful degradation | Disable LLM (no API key); request synthesis; verify skeleton produced, no error. |
| AT-O9 | No secret leakage | Build prompt; verify no API key, capsule content, or raw data in prompt string. |
| AT-O10 | Capsule-protected key | Seal API key in capsule with genesis-valid policy; verify key released when genesis valid, denied when genesis drifted. |

# Part IX

# Handoff

## 19 Dependencies

```
isls-oracle (C25)
    depends on:
      isls-types          -- foundation types
      isls-pmhd (C21)     -- mini-PMHD validation
      isls-artifact-ir (C22) -- prompt construction
      isls-forge (C23)    -- integration point
      isls-capsule (C14)  -- API key protection
      isls-store (C17)    -- pattern persistence
      isls-consensus (C5) -- 8-gate validation
    new external deps:
      reqwest             -- HTTP client for API calls (already in C20)
      serde_json          -- response parsing (already in workspace)
      syn (optional)      -- Rust syntax validation for parse stage
```

## 20 Test Count

| Phase | New Tests | Cumulative |
|---|---|---|
| Phase 1–5.1 + Genesis | 243 | 243 |
| Phase 6 (C25) | 10 | $\geq$ 253 |

## 21 Completion Criteria

Phase 6 is complete when:

1. OracleEngine synthesizes via Claude API when memory misses.
2. Memory-first lookup works and avoids LLM calls.
3. LLM output is validated through parse + constraints + PMHD + 8-gate before crystallization.
4. Validated patterns are stored in Pattern Memory.
5. Autonomy ratio is tracked and reported (M33).
6. Budget controls prevent overspend.
7. Graceful degradation works without LLM.
8. API key never appears in logs, traces, or crystals.
9. Total tests $\geq$ 253, zero warnings.
10. **25 Crates. The system that learns to stop asking.**

---

Generated for ISLS Extension Phase 6 v1.0.0.
The oracle generates. The system validates. The memory learns.
Every validated answer reduces the next question.

Deterministic envelope. Untrusted oracle. Progressive autonomy.